

Governed Memory Without Embeddings: Recall Forensics, Deterministic Retrieval, and Approval-Bound Effects in AETNAMEM

aetna000.com

Independent Researcher

aetna000@gmail.com <https://github.com/aetna000/aetnamem>

July 2026

Technical white paper; v0.3.0 core plus a content-addressed evaluation snapshot

Abstract

When a long-lived assistant answers from memory, the load-bearing question is often not “was retrieval relevant?” but “can anyone prove, later, why that memory surfaced?” We call this task *recall forensics*: from a durable database and a published evidence format, an auditor reconstructs a logged result’s ranking inputs, provenance commitment, lifecycle admissibility, and integrity evidence. We describe the governed-memory architecture in AETNAMEM v0.3.0 and evaluate additional forensics bindings in a content-addressed post-release source snapshot. Across 14 turns containing a correction, quarantine, explicit promotion, deletion, and later graph rebuild, a standard-library auditor recomputed 14/14 ranking decisions and reconstructed 68/68 returned-result pairs. Exact engine re-execution is reported only for 11/11 turns whose canonical inputs remained unchanged; three earlier turns are excluded because retaining deleted plaintext for replay would contradict the deletion contract. The evaluator also detected 75/75 selected single-row mutations. Tail truncation remained undetectable from the chain alone (0/5) and detectable with an anchored checkpoint (5/5).

The architecture behind this capability treats memory as governed local state. Canonical episodes and records carry provenance, categorical trust, lifecycle state, and append-only transition evidence on one per-subject SHA-256 chain that also records graph, retrieval, approval, and action transitions. Retrieval uses bounded SQLite FTS5 candidates and an optional, derived entity-relation graph, fused deterministically; the graph adds explainable multi-hop joins while remaining disposable and rebuildable from canonical records. No embedding model or vector database is used — a deliberate trade that avoids preserving an additional learned model, tokenizer, and vector-index artifact for replay, at the cost of some paraphrase recall. Exact-plan HMAC approvals, root-confined filesystem adapters, precondition revalidation, postcondition checks, compensation, and explicit uncertain states keep recalled information distinct from authority to act.

We state platform boundaries explicitly: on macOS the desktop launcher seals the database with OpenSSL AES-256-CBC and HMAC-SHA256 under a Keychain-held secret while the live database remains plaintext during operation; Linux and Windows rely on operating-system disk encryption. A reproducible 10,002-record laptop probe additionally demonstrates candidate-bounded recall and a two-hop graph result;

it is a scale sanity check, not a quality or million-record validation.

1 Introduction

An assistant that remembers creates a durable information system. A sentence accepted today may be recalled months later, combined with other facts, shown to a model, or used while proposing a change to the outside world. The engineering question is therefore not only “did retrieval find a relevant sentence?” It is also: who said it, was that source trusted, what did it replace, can it be deleted, why did it surface, and does it grant authority to act?

Most of those questions must be answerable *after the fact*, from evidence, by someone who does not trust the assistant’s own account of events. We call that capability *recall forensics* (Section 8) and treat it as a first-class design goal rather than a logging afterthought: each measured recall records a bounded candidate ledger and returned graph paths, the records they cite chain back to source commitments, and selected fields are digest-bound to a per-subject hash chain. A standard-library auditor can therefore reconstruct the measured decisions and detect the mutation classes evaluated in this paper, subject to explicit deletion and candidate-generation limits.

Many memory implementations begin with a document store and semantic similarity. That is useful for search, but search alone does not define durable state. Similarity does not decide whether a web page may overwrite a user’s preference; a vector does not itself express correction, quarantine, deletion, approval, or an uncertain external result. AETNAMEM starts from these state-transition questions and treats retrieval as one subsystem inside a larger control plane.

Plain-language reading. Think of the system as four ordinary things used together: a filing cabinet for the original statements, a card catalogue for finding them, a map connecting related people and facts, and a receipt book recording every important change. The map and catalogue can be rebuilt; the original statements and receipts are the durable evidence.

This paper documents the core architecture implemented in the open-source AETNAMEM release v0.3.0, at commit 69ebedea93ea95965867af5483dd9f63425b8e75 [1]. The recall-forensics bindings and evaluator are a later, content-addressed development snapshot based on commit 12925d3;

the evidence manifest, rather than that base commit alone, identifies its exact source bytes. The paper makes six scoped contributions.

1. It defines **recall forensics** as five scoped questions — attribution, provenance commitment, lifecycle consistency, decision replay, and integrity — and evaluates them with a standard-library auditor: 14/14 rankings and 68/68 returned-result pairs reconstructed, 11/11 eligible turns re-executed exactly, 75/75 selected single-row mutations detected, and three privacy-affected turns explicitly excluded from engine re-execution.
2. It defines a governed memory lifecycle in which origin determines initial trust and status, corrections supersede rather than overwrite, deletion purges payloads and emits a receipt, and recall considers only active records.
3. It combines candidate-capped lexical retrieval with an optional bounded graph traversal. The graph is a provenance-preserving, rebuildable index rather than a second source of truth.
4. It separates extraction confidence, source trust, retrieval score, entity-resolution confidence, and action outcome. None of these numbers silently becomes authority.
5. It binds memory, graph, retrieval, and effect transitions into one per-subject hash chain with externally anchorable checkpoints and crash-aware action states. The evaluated snapshot adds a versioned digest envelope for selected retrieval and record fields; it does not claim to authenticate every database column.
6. It describes the implemented platform and filesystem security boundaries precisely, including important non-guarantees.

The claim is architectural, not a claim of priority or universal superiority. The individual mechanisms—full-text search, graph traversal, provenance, hash chains, approval tokens, and compensating actions—are established ideas. The systems contribution is their composition around an explicit distinction between *information* and *authority*, together with a local implementation whose transitions can be inspected and replayed.

1.1 How to read this paper

Non-specialist readers can follow the plain-language boxes, Figure 1, the lifecycle table in Table 1, the forensics results in Table 4, and the platform matrix in Table 5. Technical readers will find the scoring equations, graph bounds, transaction model, cryptographic construction, forensics protocol, action state machine, implementation map, and reproducibility details in the main text and appendix.

1.2 Terminology: is this RAG?

Lewis et al. introduced retrieval-augmented generation (RAG) as a generator combining parametric memory with a dense vector index accessed by a neural retriever [10]. The term is now also used more broadly for almost any process that inserts retrieved material into a model prompt. Under that broad usage, AETNAMEM recall can be an augmentation source. Under the original architectural meaning, AETNAMEM is not a conventional dense RAG pipeline: it has no embedding model or vector index, and its primary responsibility is governed persistent state and effects rather than passage selection for generation.

We therefore use the precise phrase *governed memory and effect-control plane*. This does not deny that an application can use its returned records in a RAG-like prompting step. It states that retrieval augmentation is an integration point, not an adequate description of the system.

2 System Model and Invariants

AETNAMEM is an embedded Python memory engine with a loopback HTTP desktop service, a model-provider adapter, an optional MCP server, a SQLite store, and a guarded-action engine. The language model is not the memory authority. It may suggest facts or actions through defined interfaces, while deterministic host code applies policy and commits state.

The implementation is organized around the following invariants.

Invariant 1: provenance before recall. Every semantic record points to the episode from which it was derived. Every graph edge points to a record. A returned graph path can therefore be followed back to stored source text.

Invariant 2: trust controls state. A trusted user statement can enter the active set. Extracted web-page and tool-output content enters quarantine. Quarantined content cannot supersede active user memory and does not enter normal recall or graph traversal until explicit promotion.

Invariant 3: history is not overwritten. Corrections create a new record and mark the old record superseded. Forgetting tombstones and purges selected payloads, removes associated search and graph entries, and emits a digest-bound deletion receipt.

Invariant 4: derived structures are disposable. FTS indexes, entities, aliases, edges, merge proposals, and inactive graph partitions are query accelerators. Canonical records and episodes do not depend on their survival. A graph rebuild must not rewrite canonical memory.

Invariant 5: information is not authority. A recalled fact may support an action proposal, but only trusted, host-attested authorization evidence and an approval bound to the exact plan can permit an enforced external mutation.

Invariant 6: ambiguity is represented. A low extraction score causes no automatic privilege increase; uncertain entity identity waits for review; a changed filesystem precondition aborts or compensates; and an exception after dispatch becomes uncertain rather than triggering a blind retry.

Invariant 7: transitions are accountable. Semantic writes and their audit events share SQLite transactions. Audit events form a per-subject SHA-256 chain. Checkpoints can pin chain heads outside the database.

2.1 Trust boundary and actors

The principal roles are the user, the agent/model, the reviewer interface, local host code, and external content or providers. User text is not assumed correct, but it is authoritative for the user's own stated preferences. Web pages and tool results are information sources, never user authority. The reviewer token is separate from the agent token. The service is bound to loopback by default; loopback reduces network exposure but does not defend against another process acting as the same operating-system user.

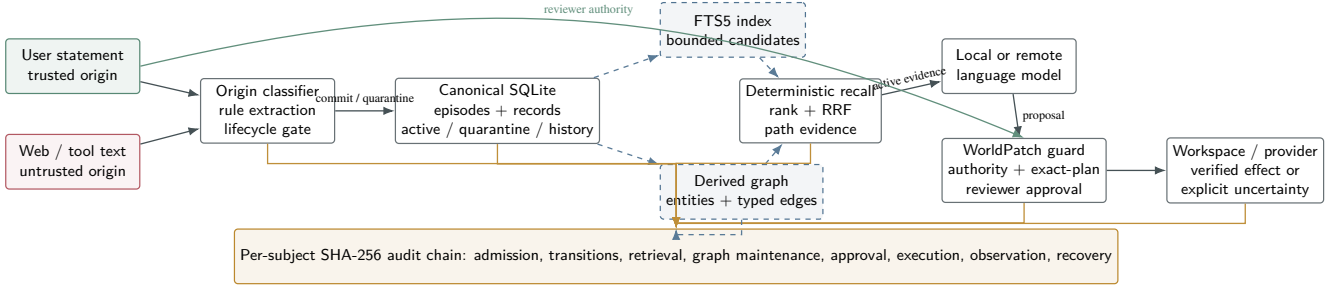


Figure 1: Implemented trust and data flow. Solid arrows carry governed state; dashed arrows are derived or reconstructable. Canonical episodes and records, action evidence, and the audit chain remain durable. The graph and FTS structures accelerate recall but do not acquire authority.

The threat model includes indirect prompt injection in fetched content, stale approvals, tampered operational rows, path traversal and symbolic-link escapes, time-of-check/time-of-use changes, partial multi-operation failure, process interruption around an external boundary, and offline theft of stored files. It does not include a fully compromised signed-in account, a malicious Python process with direct database or filesystem access, a hostile kernel, or an operator who gives the agent the reviewer secret.

Boundary Mediation assumption. The action gate controls operations routed through it. It cannot prevent separate tools, shell access, plugins, or application code from writing directly to disk. Complete mediation is therefore a deployment property, not something this Python library can impose on arbitrary neighboring software [15].

3 Governed Memory Lifecycle

The canonical data model separates raw episodes from normalized records. An episode retains the source message and origin metadata. A record stores a concise fact, its source episode, trust tier, confidence annotation, scope, lifecycle status, optional `fact_key`, and optional predecessor. This follows the database-provenance principle that a derived answer should retain both where it came from and why it exists [3].

3.1 Admission

The current extractor is deterministic and intentionally narrow. It recognizes generic forms such as “my X is Y ,” “use Y as my X ,” named possessives such as “Sarah’s airport is SEA,” explicit “remember that” statements, and avoidance statements. It emits at most one candidate from a message. A direct API caller can force an otherwise unparsed user note, but this still enters through the same record and audit path.

Origin is classified before extraction. A normal user message maps to `trusted_user` (T_u); reviewed content maps to `user_confirmed` (T_c); embedded web-page or tool-output blocks map to `untrusted_content`. The initial state function is simple:

$$\text{status}_0(t) = \begin{cases} \text{active}, & t \in \{T_u, T_c\}, \\ \text{quarantined}, & \text{otherwise.} \end{cases} \quad (1)$$

The untrusted statement is not silently discarded. It remains inspectable with provenance, but active recall and graph

Table 1: Memory lifecycle and user-visible meaning.

State	Meaning	Normal recall
Active	Accepted current memory	Eligible
Quarantined	Stored for review; source is untrusted	Excluded
Superseded	Historical value replaced by a correction	Excluded
Tombstoned	Selected payload purged; receipt retained	Excluded

spread exclude it. Promotion is a separate, explicit, audited transition to `user_confirmed`.

3.2 Correction and conflict

For structured facts, `fact_key` names a slot such as `preferred airport`; named facts use a namespace such as `sarah::preferred airport`. When a new *active* record uses an occupied single-valued slot, the old record becomes superseded and points to its replacement. No value is edited in place. A quarantined record cannot supersede an active one. This prevents a hostile page saying “your airport is X ” from replacing the user’s existing preference merely because the text matched an extraction pattern.

Some relations are multi-valued, including `works_with`, `likes`, and `avoids`; they accumulate instead of applying single-slot supersession. Duplicate detection is normalized textual equivalence, not semantic similarity. This is conservative: paraphrased duplicates may coexist, but unrelated facts are less likely to collapse silently.

3.3 Forgetting

A forget request is accepted only from a user-origin message. An empty selector is rejected instead of being interpreted as “delete everything.” Matching records are selected from active, quarantined, and superseded states; their payloads and source episodes are purged where no retained record still needs the episode. Associated graph objects and FTS rows are removed, including matching archived derived edges. The returned deletion receipt contains identifiers and digests, not the erased content, and is bound to the audit event hash.

Deletion is therefore a governed state transition, not merely

removal from a retrieval index. The audit chain proves that a deletion operation occurred without preserving the sensitive selector or deleted value in the audit payload. It cannot prove that no independent backup or external copy exists; backup retention remains an operator responsibility.

3.4 Read path

By default, `list` and `recall` expose only active records. Assistant responses and tool traffic are logged as digests but are not automatically promoted into semantic memory. This blocks a simple self-reinforcement loop in which generated text becomes a trusted fact solely because the same agent produced it.

4 Deterministic Retrieval Without Embeddings

AETNAMEM uses SQLite FTS5 as its primary lexical index [17]. FTS5 is local, transactional with the canonical store, and supports explicit row removal when a record leaves the active set. The implementation retains a token-overlap fallback when FTS5 is unavailable.

4.1 Candidate-capped record ranking

Recall does not score the complete database. For query tokens q , the store obtains up to C active FTS matches and fills unused capacity with the most recent active records, excluding duplicates. The default is $C = 200$ and the hard per-call ceiling is 2,000. This recent-record backfill means a query with no lexical match still has a bounded, deterministic fallback.

For candidate record i , the direct score is

$$S_i^{\text{record}} = 0.75T_i + 0.15U_i + 0.10R_i, \quad (2)$$

where $T_i \in [0, 1]$ is normalized FTS relevance (or query-token overlap), U_i is the source-trust weight, and $R_i \in [0, 1]$ is the record's relative recency rank within the candidate set. Current trust weights are 1.0 for `trusted_user`, 0.9 for `user_confirmed`, and 0.4 for other active tiers. Scores are rounded to six decimal places and ties are broken by creation timestamp and record identifier.

Equation 2 is a retrieval utility, not a probability that the fact is true. A record must already be active to reach this stage. A high text score cannot promote quarantined content, and a low text score does not demote or delete a record.

The retrieval event stores a query digest, returned identifiers, the algorithm and cap, and at most 50 scored candidate summaries. Query text is omitted by default. Bounded evidence avoids turning an $O(C)$ retrieval path into an unbounded audit payload.

4.2 Why embeddings are absent

Embeddings can improve paraphrase and conceptual recall, and this paper does not claim otherwise. They are omitted from the implemented trusted path for five operational reasons.

1. **Replayability.** Lexical tokens, static weights, and graph paths can be replayed without a model runtime. An embedding result also depends on model identity, model bytes, tokenizer, numerical implementation, and vector-index state.
2. **Lifecycle simplicity.** Correction and deletion update ordinary rows and FTS entries. There is no stale vector to find, no background re-embedding queue, and no dimensional migration when a model changes.
3. **Explanation.** A direct hit identifies matching terms; a graph result identifies a typed path. Nearest-neighbor distance is useful evidence of similarity but usually not a human-readable reason for a multi-hop relation.
4. **Local footprint and availability.** Recall remains available without downloading an embedding model, running a second inference path, or operating a vector extension or service.
5. **Trust composition.** Trust and quarantine are structural filters before ranking and traversal. Similarity never becomes a shortcut around admission policy.

These benefits have costs. Lexical seeding misses some synonyms, misspellings, multilingual paraphrases, and conceptual equivalence. The deterministic extractor also misses unsupported sentence forms. Direct record fallback reduces extraction loss but cannot reproduce dense semantic recall. The correct decision depends on workload; AETNAMEM chooses inspectability and small local operation for the current release.

4.3 A precise RAG boundary

If a client places returned AETNAMEM records into an LLM prompt, that client performs retrieval augmentation in the broad sense. What distinguishes the architecture is that generation does not own memory state, graph truth, trust transitions, approvals, or effect outcomes. Calling the whole system “RAG” hides the mechanisms that carry most of its safety and audit obligations.

5 The Graph as a Rebuildable Index

Lexical ranking cannot perform a join when the query and answer use different words. Suppose memory contains “my boss is Sarah” and “Sarah’s preferred airport is SEA.” A query asking “what does my boss use for flights?” may touch the first fact but not the second. The derived graph makes the shared entity explicit.

Plain-language reading. The graph is a map drawn from the filing cabinet. If the map is damaged or an extraction rule improves, aetname can redraw it from the filed records. Destroying the map does not destroy the original memory.

5.1 Derived schema and provenance

The graph stores subject-scoped entities, aliases, and typed edges. Entity and edge identifiers are content-derived. Every non-root entity records the record that introduced it; aliases cite their source records; every edge cites one record and carries that record’s trust tier, confidence annotation, status, and extractor version. The chain is therefore

$$\text{graph edge} \rightarrow \text{record} \rightarrow \text{episode} \rightarrow \text{source origin}. \quad (3)$$

Table 2: Architectural comparison. “Dense RAG” describes the common embedding-index pattern, not every system called RAG.

Concern	Conventional dense RAG pipeline	AETNAMEM v0.3.0
Primary unit	Document chunks or passages	Episodes, governed records, typed graph edges
Candidate selection	Embedding similarity, often approximate nearest neighbors	FTS5 lexical candidates plus bounded graph seeds and spread
Update semantics	Re-index or replace chunks	Active, quarantined, superseded, tombstoned transitions
Trust	Often a filter or metadata feature	Admission state; quarantined objects cannot enter ordinary traversal
Explanation	Source chunk and similarity/distance	Source record plus lexical evidence or typed traversal path
Deletion	Remove source/vector entries	Purge payloads and FTS/graph rows; retain a digest-bound receipt
External authority	Usually outside retrieval scope	Separate exact-plan approval and verified action state machine
Model dependency	Embedding model and index parameters	No retrieval model; static versioned rules and weights

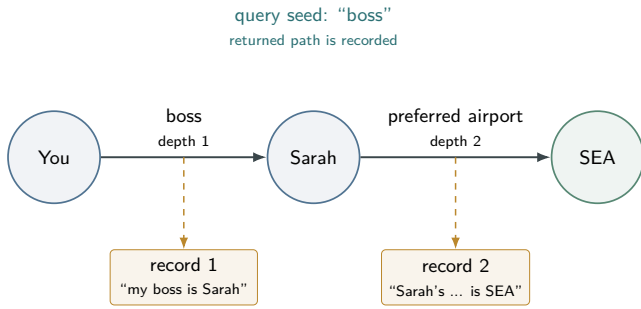


Figure 2: A two-hop recall path. Each edge cites a canonical record, and each record cites its source episode. The path explains why SEA was nominated even when the answer record did not share query terms.

The relation vocabulary is deliberately small, including `boss`, `manager`, `preferred_airport`, `works_with`, `stored_in`, and a conservative `related_to` to escape relation. A versioned rule extractor converts supported records into edges. Records that cannot be converted remain available through direct FTS recall.

Lifecycle changes propagate. An edge from a quarantined record is quarantined; an active correction supersedes the prior single-valued edge; forgetting tombstones corresponding graph objects and removes them from graph FTS. Only active entities, aliases, and edges are seedable or traversable. Consequently, untrusted content cannot become an active bridge between trusted facts merely because it connects two names.

5.2 Seed and bounded spread

Graph FTS indexes active entity names, aliases, relation labels, source names, and destination values. For a query, it returns at most $M = 16$ seeds by default. Entity and alias seeds activate an entity; an edge seed nominates that edge and continues from its destination. This directional choice avoids immediately returning through the root “you” entity, which otherwise behaves as a supernode connecting most personal facts.

Traversal uses default depth $D = 2$ and frontier cap $F = 64$. Hard limits are $M \leq 64$, $D \leq 3$, and $F \leq 256$. At depth d , an

edge reached from activation A receives

$$A_e = A W_{\text{trust}}(e) P_{\text{relation}}(e) (0.72)^d, \quad (4)$$

then a bounded recency adjustment

$$S_e = A_e (0.85 + 0.15 R_e). \quad (5)$$

Static relation priors give, for example, 1.0 to `preferred_airport`, 0.98 to `boss` and `manager`, and 0.72 to `related_to`. Trust weights are 1.0 for trusted user edges, 0.95 for user-confirmed, 0.65 for derived, and 0 for untrusted content. These are versioned heuristics, not learned parameters or calibrated probabilities.

The algorithm records seeds and paths for returned candidates, the actual visited-edge count, configured bounds, and a digest of pruned edge identifiers. Its work is bounded by seed, frontier, depth, and per-entity edge limits rather than by total graph cardinality. This is an algorithmic bound on traversal; storage lookup cost and FTS behavior still depend on database size and hardware.

5.3 Blending graph and record recall

Graph-nominated records outside the direct FTS window are loaded only if they remain active. Direct and graph rankings are combined using a weighted form of reciprocal-rank fusion (RRF) [4]. For record i ,

$$B_i = \frac{\frac{1}{60+r_L(i)} + \frac{2G_i}{60+r_G(i)}}{3/61}, \quad (6)$$

where r_L and r_G are one-based lexical and graph ranks and G_i is the graph score clipped to $[0, 1]$. Missing graph candidates contribute zero. RRF avoids pretending that the direct and traversal scores share a calibrated scale. The graph is optional and disabled by default; record-only recall remains a permanent fallback.

5.4 Consolidation, ambiguity, and cold history

A maintenance worker can backfill missing edges, propose exact same-kind entity merges, verify audit suffixes, archive old inactive edges, and run `PRAGMA optimize`. It never uses an

LLM to decide identity. Merge proposals require exact canonical or alias overlap and wait for reviewer approval. Merging marks one entity as pointing to a winner without destructively rewriting canonical records, and the decision can be reverted.

Only superseded and tombstoned *derived edges* move to subject/year SQLite partitions. Each partition digest is registered in the primary database and checked before read. Canonical records, episodes, and audit events stay in the primary database. Missing or modified partitions are detectable and can be rebuilt, subject to retained canonical history. Partition digests provide integrity detection, not confidentiality.

The published scale regimes of one SQLite file through roughly one million edges and inactive partitions into the multi-million range are engineering targets, not validated capacity claims. The evidence in this paper covers 10,002 records only (Section 11).

6 Confidence, Trust, and Ambiguity

Systems often place one “confidence score” on an item and then use it for unrelated decisions. AETNAMEM instead keeps five concepts separate (Table 3). This separation is central to its behavior.

6.1 Language ambiguity

The rule extractor favors abstention. Unsupported phrasing creates an episode but no semantic record unless the caller explicitly requests a forced note. Questions are not interpreted as assertions. A user can inspect or correct memory instead of having an opaque model infer a fact. This loses recall coverage but makes extraction failures reproducible.

6.2 Source ambiguity

A user turn containing marked web-page or tool-output content is classified by the embedded origin, because that is the text the extractor would otherwise mistake for the user’s own statement. This is a conservative boundary. It depends on the host adapter labeling external content correctly; unlabeled hostile text presented as a direct user message cannot be distinguished by intent alone.

6.3 Fact conflict

Recognized slots use explicit supersession. Unstructured facts with no `fact_key` do not silently replace one another. This means some conflicts remain visible as multiple records. The design prefers a reviewable ambiguity over a guessed destructive update.

6.4 Entity ambiguity

Entity resolution is proposal-only. Candidates must share an exact normalized canonical name or alias and the same entity kind. The graph records evidence record identifiers and a heuristic confidence, then waits for a reviewer to merge or keep the entities separate. A merge changes a reversible pointer in the derived graph; it does not rewrite source records.

6.5 Effect ambiguity

The action engine distinguishes a rejected plan, a known precondition failure, a verified external result, an exception after dispatch, and a process interruption during an external call. The last two are not reduced to a numeric confidence. They become uncertain or `recovery_required`, because a generic engine cannot infer whether the provider completed an operation. This avoids duplicate effects caused by optimistic retries.

Plain-language reading. A score answers “how should this candidate be ordered?” Trust answers “may it participate?” Approval answers “may this exact change run?” Verification answers “did the outside world reach the expected state?” These are different questions.

7 Auditability and Fault Behavior

Memory mutation, retrieval, graph maintenance, approvals, and action outcomes append events to a per-subject audit log. An event contains its identifier, subject, type, timestamp, actor, optional session/turn/record references, structured payload, and predecessor hash. With canonical JSON serialization, the chain is

$$H_n = \text{SHA256}(\text{Canon}(E_n \setminus \{H_n\})), \quad (7)$$

$$E_n.\text{prev_hash} = H_{n-1}.$$

This construction follows the general lineage of hash-linked timestamp and secure-log structures [8, 16]. It makes an in-place edit, insertion, or interior deletion detectable by verification from the chain start.

7.1 Atomic semantic transitions

The SQLite connection uses explicit transactions. Public memory operations open `BEGIN IMMEDIATE`; nested store calls join the outer transaction. Creating an episode, inserting or superseding a record, updating its graph objects, and appending the corresponding audit events therefore commit or roll back together. Immediate transactions also serialize reading the current subject head and appending the next event, preventing two writers on that connection from deriving competing successors.

File-backed stores request SQLite write-ahead logging (WAL). WAL appends commits to a separate log and later checkpoints them into the main database [18]. WAL supports crash recovery and reader/writer concurrency, but the `-wal` file is part of live persistent state. A backup made while the service runs must use a SQLite-aware method or include the required files; copying only the main database can omit committed data.

7.2 Anchoring and verification

A hash chain stored in the same database cannot by itself detect complete replacement or a recomputed chain after tail truncation. AETNAMEM checkpoints therefore export, for each subject, the event count, sequence, and current hash, plus a digest over the checkpoint document. The operator must place that checkpoint in an independent location such as append-only object storage, a transparency service, or a timestamping service.

Table 3: Confidence-related values and their authority. Values shown are the implemented v0.3.0 defaults.

Mechanism	Range or values	Meaning	What it may decide
Extraction confidence	0.90 slot forms; 0.88 named slot; 0.85 generic remember; 0.80 avoidance; 0.70 forced note; untrusted capped at 0.30	Heuristic annotation from a deterministic parser	Stored as provenance; does not activate untrusted content
Trust tier	trusted_user, user_confirmed, untrusted_content, derived	Categorical origin and review status	Initial lifecycle state and traversal eligibility
Record retrieval score	[0, 1] weighted utility in Eq. 2	Query relevance, trust prior, and relative recency	Ordering and optional <code>min_score</code> filtering only
Graph score	Bounded activation in Eq. 4 and fused rank in Eq. 6	Strength of a typed path from lexical seeds	Candidate nomination and ordering only
Merge confidence	0.99 canonical/canonical; 0.96 canonical/alias; 0.93 alias/alias	Strength of an exact same-kind identity proposal	Display and audit evidence; never automatic merge
Action outcome	verified, uncertain, recovery required, compensated, partial	Observation of external state and execution history	Whether commit is claimed, fenced, or requires review

Verification checks both the chain and containment of pinned events.

At scale, a local cache records the last verified head. Incremental verification first confirms that cached event and its hash still exist, then verifies only the suffix. This improves routine cost but is explicitly not a trust anchor: an attacker able to replace the database can replace the cache. External checkpoints remain necessary.

7.3 Three different meanings of fault tolerance

The phrase “fault tolerant” is too broad unless the fault is named.

1. **Derived-index recovery.** FTS and graph state can be cleared and rebuilt from canonical records. Archived inactive-edge partitions are digest checked. This contains extractor and index corruption without making the graph authoritative.
2. **Crash consistency.** SQLite transactions and WAL protect local committed state. The macOS seal path checkpoints before producing a new encrypted snapshot. External actions use persisted transition states so interruption is visible.
3. **Tamper evidence.** Hash links expose modification relative to a trusted chain start; external checkpoints expose tail loss relative to a prior anchor.

These mechanisms do not provide distributed replication, continued availability after disk loss, Byzantine consensus, or automatic repair of a maliciously recomputed unanchored database. Backups remain required.

7.4 External-boundary recovery

No database transaction remains open across a provider or filesystem call. Before dispatch, the action and attempt state are persisted as executing. If the call raises, the operation is marked uncertain; if a process restarts with a committing or compensating transaction, in-flight operations become uncertain and the transaction becomes `recovery_required`.

Provider-specific idempotency lookup or authoritative state inspection may later resolve it. Generic code does not retry blindly.

For a multi-operation plan, verified earlier effects can be compensated in reverse. Compensation runs only if the observed state still matches the engine’s prior verified result; otherwise it refuses to overwrite a later change. The final state can consequently be compensated, partial, or uncertain, rather than an inaccurate all-or-nothing claim. This resembles saga-style recovery while retaining explicit verification evidence [6].

8 Recall Forensics

Memory evaluations usually ask whether retrieval found a relevant item. A different question matters after deployment: *given a particular result shown at a particular turn, what durable evidence explains why it appeared, where it came from, and whether the evidence was later altered?* We call this task *recall forensics*. We have not identified a directly comparable assistant-memory evaluation in the systems reviewed in Section 13; this is a scoped observation, not a claim that no other system can implement the task.

Plain-language reading. If the assistant said something surprising last Tuesday, the database should show which accepted statement contributed to that result, who supplied it, which score or connection selected it, and whether the recorded trail still verifies. If the source was later deleted, the system should prove the deletion rather than retain the words for convenience.

8.1 Task definition

The auditor receives the database, the evidence-format specification, and a content-addressed source manifest. The reconstruction code does not call the engine’s ranking, graph, lifecycle, or audit helpers. For a logged result X at recall turn T , it asks:

- F1 Attribution.** Recompute the text, trust, and recency components, Eq. 2, graph fusion when present, rank order, threshold decision, and result-limit decision from the bounded candidate ledger. A graph-nominated result must also carry a typed path whose objects were active according to graph events before T .
- F2 Provenance commitment.** Resolve record $\rightarrow$ admission $\rightarrow$ episode $\rightarrow$ origin and verify the chained content commitments. If a later deletion purged the source text, verify its prior digest and deletion transition without pretending the plaintext remains available.
- F3 Lifecycle consistency.** Replay admission, quarantine, explicit promotion, supersession, consolidation, and forgetting up to T . A returned record must be active at that point; the final row's status, source, trust tier, confidence, scope, and episode link must agree with the complete transition history.
- F4 Replay.** First, replay the recorded ranking decision for every turn. Separately report exact engine re-execution only for turns whose canonical inputs still exist unchanged. A turn before a later privacy deletion is deliberately ineligible for engine re-execution.
- F5 Integrity.** Test named mutation classes against copies of the database and report each class. This supports only the tested adversary model, not the universal statement that every possible forged field must be detected.

Deterministic retrieval reduces the artifact set needed for F4; it does not make embedding retrieval unreplayable in principle. A dense retriever can be replayed if the exact model and tokenizer bytes, vector-index state, preprocessing, numerical environment, dependencies, and code are preserved. The narrower advantage here is that replay does not require preserving an additional learned-model artifact.

8.2 Evidence bindings

Candidate summaries and paths live in a retrieval-event row rather than directly in the audit chain. The evaluated post-v0.3.0 development snapshot therefore stores a versioned evidence envelope and places its digest in the chained `memory.recall` event:

$$D_{\text{ret}} = \text{SHA256}(\text{Canon}(v, i, s, u, q, q_{\text{sha}}, C, R, G)), \quad (8)$$

where v is the format version, i the retrieval identifier, s and u the subject and session, q the retained query (empty by default), q_{sha} its digest, C the bounded candidate ledger, R the ordered returned set, and G the graph and replay parameters. The envelope records one-based final and lexical ranks, raw text-normalization inputs, trust tier, recency rank and denominator, base score, graph rank and strength, and the typed path for each returned graph nominee. The first 50 ranked candidates are retained, plus every returned candidate when `limit` exceeds 50.

Admission events bind record content and the lifecycle metadata used by the auditor: source type, initial trust and status, fact key, extraction confidence, scope, and episode identifier. Promotion and later transitions remain separate chained events. Episode ingestion already binds the original message digest. These commitments detect direct changes to the selected retrieval, record, and episode fields while preserving the rule that a tombstone removes content.

Table 4: Recall-forensics results for the content-addressed snapshot.

Measure	Result
Untampered chain, digest, lifecycle, turn failures	0
F1 ranking decisions recomputed	14/14
F1 returned-result attribution	68/68
historical graph paths valid	14/14
paths absent from current graph	1
F2 provenance commitments complete	68/68
later-deleted source payloads unavailable	1
F3 lifecycle admissible at recall	68/68
explicit promotion transition verified	yes
F4 stable returned sets re-executed exactly	11/11
complete candidate ledgers exact	11/11
historical turns excluded from re-execution	3
F5 selected single-row mutations detected	75/75
tail truncation, chain only	0/5
tail truncation, anchored checkpoint	5/5
Quarantined / post-deletion results returned	0 / 0
Forgotten content rows remaining	0
Auditor reconstruction time	1.154 s

The forensics experiment is not attributed to release commit 12925d3 alone. That commit is only the snapshot's base. The evidence file contains SHA-256 digests for 42 source files, including the evaluator and the modified package, plus a digest over the manifest. Reproduction therefore targets those bytes rather than an incorrect clean-commit claim.

8.3 Protocol

The harness is stored at:

`paper/scripts/run_recall_forensics_eval.py`

It creates nine initial scenario records, including an unreviewed web claim, a second web-derived record that is explicitly promoted, a correction target, a secret later deleted, and a two-hop relation, plus 1,000 filler records. It executes three recall turns, then applies a correction and deletion, executes eleven stable-state turns, and finally rebuilds the derived graph. Across 14 turns it varies limits from 3 to 7, exercises a non-null threshold, and enables graph recall in nine turns.

The standard-library reconstruction verifies all turns after the later mutations and graph rebuild. Engine re-execution uses the logged `limit`, `min_score`, candidate cap, and graph-mode fields; it is restricted to the eleven stable-state turns. The three earlier turns remain decision-reconstructable but are excluded from engine re-execution because a correction or deletion changed their canonical inputs. Fifteen selected single-row mutation classes cover audit payload and event changes, retrieval scores, returned sets, parameters, query and subject fields, record content and lifecycle metadata, and episode text. Five trials are run per class.

8.4 Results and boundaries

Table 4 reports the pinned run. The auditor independently recomputed score arithmetic, ranking, thresholds, and limits for

all 14 turns, and reconstructed attribution, provenance commitments, and lifecycle admissibility for all 68 returned-result pairs. All 14 typed graph paths were valid against historical graph events. One old path no longer resolved in the current derived graph after deletion and rebuilding, demonstrating why historical evidence must not depend on current index rows. The explicitly promoted record surfaced only after its promotion event.

All eleven turns whose canonical inputs remained stable reproduced both the returned identifiers and complete bounded candidate ledgers. The three prior turns were not counted as exact engine replays: retaining deleted plaintext solely to make a benchmark replayable would violate the deletion contract. For one returned-result pair, the later-deleted source is represented by its ingestion digest, origin metadata, and deletion event; the source words are correctly unavailable.

All 75 selected single-row mutations were detected. This is mutation coverage, not a proof against arbitrary database rewriting. A coherent attacker who rewrites the complete file and recomputes an unanchored chain remains outside the file-only trust model. Tail truncation was correspondingly undetectable from the chain alone (0/5) and detected against an anchored head (5/5).

Further limits are material. The candidate ledger is bounded; the auditor recomputes selection within that sealed ledger but does not independently regenerate every FTS candidate from historical corpus state. With query-text retention disabled, it can confirm a proposed query against the digest but cannot discover an unknown query. The workload is synthetic and single-host, and the results establish neither fact truth nor general retrieval quality. They establish what the measured evidence can and cannot reconstruct.

9 Storage and Encryption by Platform

Encryption claims must identify both platform and process state. AETNAMEM v0.3.0 does not use SQLCipher and does not provide uniform live database encryption. Its implemented behavior is summarized in Table 5.

9.1 macOS sealed-database construction

On first use, the launcher obtains a secret from the aetname/database-key login-Keychain item or generates 32 random bytes represented as 64 hexadecimal characters. macOS Keychain is intended for small secrets and encryption keys, with access mediated by the `securityd` service and Keychain access controls [2].

At startup, if a sealed file exists, the manager computes

$$M = \text{HMAC}_{\text{SHA256}}(K, C) \quad (9)$$

over the encrypted bytes C and compares it in constant time with the sidecar value before decryption. It then invokes `OpenSSL enc -aes-256-cbc -pbkdf2 -salt` and writes the plaintext SQLite database into a new temporary directory. OpenSSL documents that `-pbkdf2` uses a 10,000-iteration default unless `-iter` is supplied, and SHA-256 is the default passphrase digest in current releases [12]. The application does not currently pin these parameters in a versioned envelope.

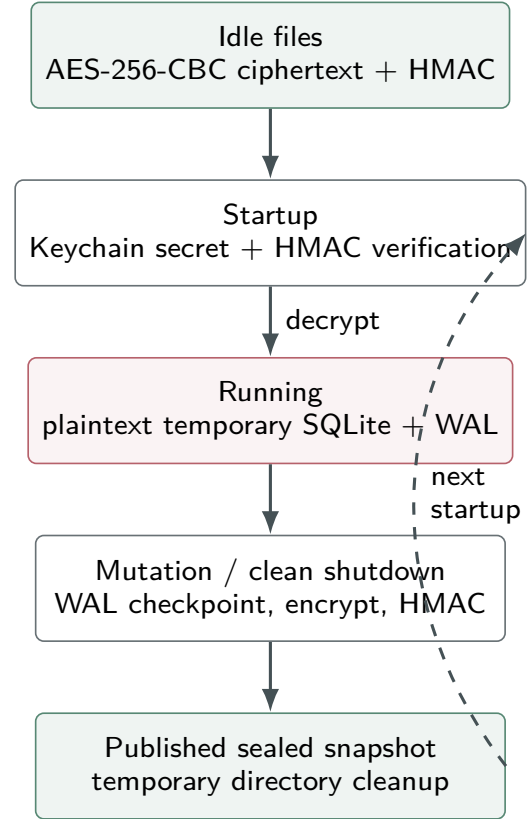


Figure 3: macOS sealed mode. The bold boundary is encrypted at idle; the runtime SQLite and WAL state are plaintext to the signed-in process while the service is open.

During operation, SQLite reads and writes the temporary plaintext file. A mutation hook first requests a truncating WAL checkpoint, then creates a new encrypted temporary file, computes its HMAC, replaces the sealed file, and writes the HMAC sidecar. Sealing is mutation-triggered and rate-limited to at most once per default 15-second interval; it is not an independent periodic timer. Pending dirty state is force-sealed on a clean service shutdown before the database is closed and the temporary directory is removed.

The construction provides useful confidentiality against an attacker who obtains only idle sealed files and not the Keychain secret, and HMAC provides ciphertext integrity and authenticity under that secret [9]. It has significant limits:

- CBC encryption plus a separate HMAC is not a single atomic authenticated-encryption envelope. The sealed file and HMAC sidecar are replaced separately, so a crash can leave a mismatched pair that requires backup recovery.
- The same high-entropy secret is used as the OpenSSL passphrase and HMAC key. A hardened format should derive independent encryption and authentication keys with explicit domain separation.
- The live database is plaintext while the service runs. An unclean process or machine failure can leave temporary plaintext until operating-system cleanup and can lose changes after the last successful seal.
- Malware or another process running as the signed-in user may read the live file or request the Keychain item, depending on operating-system controls. This is not protection against an already compromised account.
- Key recovery and backup confidentiality remain user re-

Table 5: Default storage and residual protection boundaries. Operating-system encryption is effective only when enabled and correctly managed.

Platform and file	Application protection and residual boundary
macOS launcher \$HOME/Library/ Application Support/ aetnamem/ memories.db.enc	AES-256-CBC sealed snapshot with PBKDF2, random salt, HMAC-SHA256, and a login-Keychain secret. Runtime SQLite is plaintext; explicitly enabled graph archives are plaintext; FileVault remains recommended.
Linux launcher \$XDG_DATA_HOME/ aetnamem/ memories.db or \$HOME/.local/share/ aetnamem/ memories.db	Ordinary SQLite with no application wrapper. An encrypted home, LUKS/dm-crypt, or equivalent is required for at-rest confidentiality.
Windows launcher %LOCALAPPDATA%/ aetnamem/ memories.db	Ordinary SQLite with no application wrapper. BitLocker or equivalent is required for at-rest confidentiality.
Direct service \$HOME/.aetnamem/ memories.db	Plaintext by default; macOS --encrypted-db opts into sealed mode. The configured path determines the actual boundary.

sponsibilities. The sealed file is unusable after loss of its Keychain secret.

This is *application-level idle sealing*, not transparent database encryption or continuous at-rest protection.

9.2 Windows and Linux

The Windows and Linux launchers store ordinary SQLite files. No application key is generated and no AETNAMEM cryptographic wrapper is applied. On Windows, BitLocker can protect an entire volume against offline theft when enabled; Microsoft documents AES-based volume encryption and TPM/key-protector options [11]. On Linux, common deployments use dm-crypt with a LUKS on-disk format [5]. These controls operate below SQLite and can protect database, WAL, temporary, archive, and workspace files together, but normally expose plaintext to processes after the user unlocks and mounts the system.

9.3 Archives, workspaces, and backups

The assistant workspace is separate from the memory database. Files there are governed by the action adapter but are not encrypted by AETNAMEM. Graph history partitions are also plaintext SQLite on all platforms unless the containing filesystem supplies encryption. Automatic graph archival defaults off in macOS sealed mode; explicit enablement requires separately protecting the archive directory.

Backups must include both memory and workspace if complete recovery is desired. A stopped-service copy is simplest. On live WAL databases, copying only memories.db is unsafe because committed frames may remain in memories.db-wal [18]. macOS recovery requires the sealed file, its HMAC sidecar, and the Keychain recovery secret.

10 Approval-Bound External Effects

Recall can inform an action proposal, but a memory record is not itself permission. AETNAMEM routes model-requested workspace mutations through a guarded action engine whose unit is a canonical WorldPatch. The implementation follows the reference-monitor principle of a small, inspectable mediation point, subject to the deployment boundary stated below [15].

10.1 Information-to-authority separation

An enforced non-read-only operation must carry an authorized_by evidence reference. Policy accepts only trusted_user or user_confirmed authority and, by default, requires the reference to be host-attested. Untrusted records may inform a proposal but cannot authorize it.

The agent-facing dispatch endpoint cannot claim user_message provenance, construct authority inline, or create an authority object. The reviewer endpoint creates host authority from the user's task. Thus a model cannot convert a recalled sentence or a web-page instruction into authority merely by labeling it differently.

10.2 Exact-plan approval

Preparing a patch calls each adapter's deterministic prepare method and records operation, argument, preview, precondition, adapter-manifest, dependency, and evidence digests. The plan hash covers these values together with subject, actor, mode, transaction identifier, format version, and policy hash:

$$P = \text{SHA256}(\text{Canon}(\text{WorldPatch})). \quad (10)$$

The reviewer authority issues an HMAC-SHA256 approval over transaction identifier, P , approver label, issue and expiry times, and a random nonce. The default validity window is 900 seconds; keys shorter than 32 bytes are rejected. Approval applies to one exact plan, not to an informal instruction or future variant. HMAC verification uses constant-time comparison [9].

Before the first effect, commit recomputes persisted plan and approval digests, verifies the audit chain and approval, checks that adapter manifests did not change, and revalidates the complete patch. Each operation is revalidated again immediately before execution. A changed precondition aborts before the first effect or triggers compensation for already verified effects.

10.3 Root-confined filesystem adapter

The reference filesystem adapter intentionally exposes only UTF-8 write_text and delete_file. For an agent-origin operation it enforces:

- a relative path under one configured workspace root;
- rejection of absolute paths, root escapes, every symbolic-link path component, symbolic-link targets, and non-regular files;
- an existing parent directory for guarded writes;
- a before-image containing existence, kind, SHA-256, mode, and content needed for compensation;
- compare-before-write revalidation against that snapshot;

- temporary-file write, `fsync`, and `atomic os.replace`;
- independent postcondition read of existence, digest, and mode;
- compensation only if the file still matches the verified after-state, followed by verification of the restored before-state.

The adapter classifies this behavior as `verified_compensatable`, never as an exact transaction. File watchers, synchronization clients, build hooks, and other observers can cause effects that restoring bytes does not reverse.

10.4 What “disk access requires approval” means

The exact implemented claim is narrower than the slogan:

- **Model-requested workspace writes and deletes through the registered broker** require trusted authority, reviewer approval, reviewer-triggered commit, and verification.
- **Workspace reads** are available to the agent role within the configured root and do not require per-read approval. This is a confidentiality boundary operators must understand.
- **A dashboard save** is a reviewer-token action representing the human editing their own workspace. It bypasses staging but remains path-confined, size-limited, and audited as `user.file_saved`.
- **Internal database writes** follow memory policy and transactions, not the external-action approval workflow.
- **Other software or direct process access** can bypass the adapter. Strong deployment requires withholding shell/direct filesystem tools or using operating-system sandboxing so all agent mutations are routed through the control service.

The service exposes distinct random bearer tokens. Agent routes may inspect state and stage tools; reviewer routes configure providers, create authority, approve, commit, deny, decide graph merges, and directly edit workspace files. Tokens are compared in constant time. The HTTP server accepts loop-back addresses only and is single-threaded around one SQLite connection. It has no TLS and is not suitable as a public network service. Any process running as the same user that obtains the reviewer URL fragment or token can exercise reviewer authority.

Plain-language reading. The assistant may prepare a precise change and show what it would affect. The human approves that exact version. If the file changes before execution, the approval is not stretched to cover the new situation. If the system cannot prove what happened, it says “uncertain” instead of pretending success.

11 Implementation Evidence and Scale Probe

This paper evaluates implementation consistency, the recall-forensics task of Section 8, and one bounded-recall scenario. It does not claim state-of-the-art answer quality. The committed scale evidence in `paper/data/governed-memory-probe.json` comes from release v0.3.0. The forensics evidence in `paper/data/recall-forensics.json` comes from the content-addressed post-release snapshot identified by its embedded 42-file digest manifest.

Table 6: 10,002-record local probe. Times are milliseconds.

Measure	Direct	Graph
Median recall	17.139	24.799
p95 order statistic	17.417	25.394
Target rank in top 10	absent	2
Candidate cap	200	200 + bounded spread
Logged candidate summaries		50
Graph edges visited	–	2

11.1 Questions and method

The probe asks three narrow questions: (Q1) does direct scoring remain capped after an older relevant record is surrounded by 10,000 newer records; (Q2) can the graph recover a two-hop target that direct top-10 recall omits; and (Q3) what local latency overhead does graph recall add in this scenario?

The harness creates an in-memory SQLite store, remembers “my boss is Sarah” and “Sarah’s preferred airport is SEA,” then adds 10,000 synthetic slot records. It issues “What does my boss use for flights?” with a result limit of 10 and a direct-candidate cap of 200. Three warm-up pairs precede 25 measured direct/graph pairs. Latencies exclude ingestion and graph construction. The p95 reported by the script is an order statistic, not an interpolated estimate.

The host was an Apple M2 with 16 GiB RAM, macOS Darwin 25.5.0, and Python 3.12.2 running as an x86-64 process. The source commit was 69ebede93ea95965867af5483dd9f63425b8e75.

Ingestion of all 10,002 records took 5,621.290 ms in this in-memory run. Graph recall added 7.660 ms to the median (about 45% relative overhead) and recovered the target at rank 2. Direct recall omitted it because FTS reached the “boss” edge but the target answer record shared no query term and was older than the 200-record recent fallback window. The result supports the intended multi-hop mechanism for this construction; one query cannot establish general retrieval quality.

11.2 Invariant tests

On the evaluated source tree, `python3 -m pytest -q` completed 155 tests (the final timing is reported in the evidence provenance file). The suite includes focused checks for retrieval-envelope binding, score inputs, result limits above the 50-row diagnostic window, and graph rebuild determinism, in addition to admission policy, quarantine and promotion, correction and deletion, FTS and graph lifecycle, path recall, reversible merges, archive digests, incremental audit verification, approvals, precondition changes, uncertain outcomes, compensation, root confinement, service roles, and macOS sealing behavior. Passing unit and integration tests demonstrates conformance to the encoded cases, not absence of security defects.

11.3 What supports scale, and what is unproven

Three implementation choices prevent obvious linear work in the hot recall path: direct candidate scoring is capped; graph traversal has explicit seed/frontier/depth bounds; and only in-

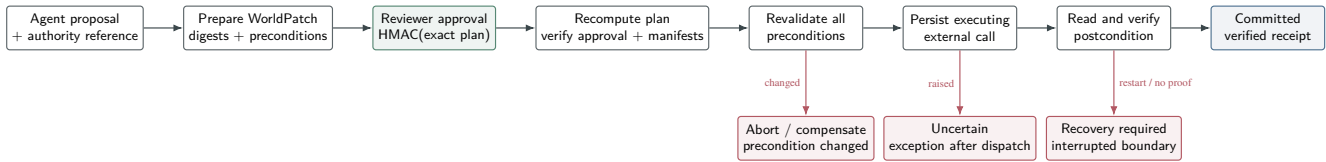


Figure 4: Guarded mutation path. Approval is bound to the canonical plan; execution occurs outside the SQLite transaction; observations determine whether the engine reports committed, compensated, partial, uncertain, or recovery-required.

active derived edges are eligible for cold subject/year partitioning. FTS and graph row maps permit targeted updates and deletions. Incremental audit verification avoids rehashing a validated prefix during routine checks.

These properties make larger stores technically plausible without changing canonical semantics. They do not establish a supported maximum. This release has no committed million-record benchmark, no sustained concurrent-write test, no long-running archive churn study, and no representative multilingual or paraphrase corpus. SQLite file size, FTS selectivity, cache pressure, audit volume, backup duration, and maintenance time remain empirical constraints. Claims beyond the 10,002-record probe are design projections.

11.4 Reproduction

From the repository root:

```
python3 -m pytest -q
python3 paper/scripts/run_governed_memory_probe.py \
  --records 10000 --iterations 25 --candidate-cap 200
python3 paper/scripts/run_recall_forensics_eval.py \
  --filler 1000 --seed 20260716
cd paper && make aetname-governed-memory.pdf
```

Timing results should be expected to vary across hosts and runs. Rank, cap, path, and visited-edge assertions are the more reproducible outputs. The recall-forensics counts in Table 4 are deterministic for the file-digest manifest, fixed workload, and seed; the reconstruction time is a single-host observation.

12 Security Analysis and Limitations

The architecture reduces several failure modes by making them explicit state transitions. It does not create a universal security boundary around an arbitrary agent.

12.1 Memory poisoning

Marked web and tool content cannot enter active memory, supersede a trusted slot, seed normal graph traversal, or authorize an effect without promotion. This addresses a durable form of indirect prompt injection [7]. The boundary depends on correct origin labeling by the host. A compromised adapter that presents remote text as a direct user message defeats the classification premise. A user can also explicitly promote false content; the audit log records that choice but cannot determine truth.

12.2 Retrieval quality

No embedding index means no learned semantic neighborhood, but it also means synonym and paraphrase misses. The

extractor and relation vocabulary are English-oriented and deliberately incomplete. Graph edges are only as accurate as their cited records and deterministic parse. Path explanations expose this lineage but do not guarantee that a relation is factually correct.

Record, graph, and merge scores are heuristic and uncalibrated. They should not be displayed as probabilities of truth. Current weights were chosen as deterministic operational defaults, not fitted or validated against a broad relevance corpus.

12.3 Audit integrity

The audit chain is unkeyed SHA-256. It detects modification relative to a trusted head, but an attacker with complete database control can rewrite events and recompute an unanchored chain. External checkpoint custody is therefore part of the security model. Timestamps are local wall-clock values and are not trusted time unless a checkpoint is externally timestamped. Audit digests also do not prove that every external effect passed through the system.

12.4 Encryption and local compromise

Only the macOS launcher implements application-level idle sealing. Windows and Linux database files are plaintext unless protected by the operating system. macOS runtime plaintext, shared key material, unpinned PBKDF2 parameters, non-atomic ciphertext/sidecar replacement, plaintext archives, and user-managed recovery are documented weaknesses. Provider API keys use macOS Keychain when configured there; on other platforms the desktop service holds entered keys in process memory for the session. Environment-provided secrets remain visible according to operating-system process and shell rules.

The current sealed format should be regarded as a compatibility format, not the final cryptographic design. A future version should define a versioned envelope, derive independent keys, use an authenticated-encryption library or standard container, make snapshot publication atomic, and add equivalent application-level behavior on Windows and Linux. Such work requires migration and recovery tooling, not only changing a cipher name.

12.5 Action mediation

The guarded adapter narrows model-requested writes, but the agent role can read workspace files. Reviewer-token saves intentionally bypass staging. The loopback server has no TLS, and bearer-token security depends on local secrecy. A process with direct filesystem, database, debugger, or reviewer-token access can bypass or impersonate the gate. Operating-system

sandboxing and least-privilege deployment are necessary for complete mediation.

An HMAC approval proves possession of one shared reviewer secret, not a person's legal identity. The approver label is asserted by the reviewer-side process. For multi-user or regulated deployments, asymmetric signing, hardware-backed keys, durable identity, revocation, and policy administration would be required.

12.6 Fault and availability limits

Rebuildable indexes improve recoverability; they do not replace backup. Partition digests detect a changed archive but cannot restore it. SQLite WAL and transactions provide local crash semantics, not replicated availability. The action engine records uncertainty but cannot generically resolve a provider that offers neither idempotency lookup nor authoritative post-condition reads.

12.7 Privacy

Canonical episodes contain raw user messages. Records contain normalized personal facts. Although query text is omitted from retrieval events by default, identifiers, timestamps, scores, and query digests can still be sensitive metadata. Subject identifiers are not anonymized by the storage layer. Operators must apply filesystem permissions, disk encryption, retention policy, and backup controls appropriate to the data.

Boundary No formal proof. The invariants in this paper are implementation contracts supported by tests and source inspection. They have not been machine-verified, independently audited, or proven against all executions.

13 Related Systems Perspective

Long-term LLM memory work commonly studies storage management, reflection, and answer quality. Generative Agents combines observations, reflection, and planning [14]; MemGPT treats memory tiers as an operating-system-inspired context-management problem [13]; MemoryBank and Long-MemEval study persistent conversational memory and its evaluation [20, 19]. AETNAMEM is complementary: its central object is a governed transition with provenance and authority boundaries, not a new language-model memory capacity result.

Dense RAG combines a neural retriever and non-parametric passage store with a generator [10]. AETNAMEM instead uses lexical and relational indexes over canonical personal state. Its graph resembles a small knowledge graph, but it is intentionally subordinate to source records. Entity resolution is reviewed because an incorrect identity merge can corrupt many later paths.

The audit design draws on provenance and hash-linked log research [3, 8, 16]. The action design draws on complete mediation and compensating transactions [15, 6]. The distinctive element is not a new primitive but a shared evidence plane linking memory admission, retrieval reasons, approval scope, external observations, and recovery state.

Recall forensics (Section 8) is where that composition creates a useful evaluation target. Retrieval research logs relevance for training or debugging, and secure-logging research

protects generic events; among the assistant-memory systems reviewed here, we did not find an evaluation combining digest-bound score inputs, typed historical paths, lifecycle replay, deletion-aware provenance commitments, and independent mutation testing. This is a literature-scope statement, not evidence of universal priority. Embedding-based memories can also be replayed when their exact model, tokenizer, vector index, numerical stack, dependencies, and code are preserved. AETNAMEM's narrower property is replay without that additional learned-model artifact.

14 Conclusion and Potential

AETNAMEM treats personal memory as governed local state. Original episodes and semantic records remain canonical; FTS and graph structures make them findable; trust and lifecycle state decide participation; audit events explain change; and a separate approval path decides whether information may become an effect. The language model can be local or remote and can change without requiring a new retrieval model or re-embedding the database. The desktop default uses the lightweight qwen3:1.7b Ollama model when available, but memory extraction, retrieval, trust, and audit do not depend on that model.

The system is therefore more accurately described as a *governed memory and effect-control plane* than as a conventional embedding-based RAG implementation. A client may use recall to augment generation, but generation remains downstream of explicit state and policy.

Its practical potential follows from testable properties rather than a novelty adjective: bounded candidate work can keep hot recall manageable as canonical history grows; a derived graph can answer some joins that lexical search cannot; rebuildability contains index mistakes; quarantine prevents unreviewed content from becoming a trusted bridge; and recall forensics gives users and auditors a way to reconstruct the measured retrieval decisions. The experiment recomputed all 14 rankings and all 68 returned-result pairs, while explicitly excluding three privacy-affected turns from engine re-execution and identifying one deleted source whose plaintext was unavailable. The 10,002-record probe supports the scale mechanisms at modest size. Larger claims require larger and more representative evidence.

The highest-priority technical work is correspondingly concrete: broaden deterministic extraction and query normalization without weakening provenance, establish representative retrieval-quality and million-record benchmarks, harden and unify the encrypted storage format, add operating-system sandbox profiles for complete action mediation, and independently review the cryptographic and action-state implementations. Embedding retrieval is not part of this design or roadmap; improvements should first preserve the deterministic, deletable, and path-explainable properties documented here.

Plain-language reading. For a user, the intended result is simple: the assistant remembers what was actually accepted, shows where a memory came from, keeps suspicious material separate, asks before changing files, and admits when it cannot prove what happened. The technical machinery exists to make those ordinary expectations enforceable and inspectable.

References

- [1] aetna000.com. aetnamem: Auditable memory, governed graph recall, and guarded actions for agents, 2026. Commit 69ebe-dea93ea95965867af5483dd9f63425b8e75.
- [2] Apple Inc. *Keychain Data Protection*. Apple Platform Security, 2024. Accessed 2026-07-16.
- [3] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, pages 316–330. Springer, 2001.
- [4] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- [5] cryptsetup contributors. cryptsetup and LUKS: Open-source disk encryption, 2026. Project and LUKS specification index; accessed 2026-07-16.
- [6] Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987.
- [7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023.
- [8] Stuart Haber and W. Scott Stornetta. How to time-stamp a digital document. *Journal of Cryptology*, 3(2):99–111, 1991.
- [9] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. HMAC: Keyed-hashing for message authentication. RFC 2104, Internet Engineering Task Force, 1997.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [11] Microsoft. *BitLocker Frequently Asked Questions*. Microsoft Learn, 2026. Accessed 2026-07-16.
- [12] OpenSSL Project. *openssl-enc: Symmetric Cipher Routines*. OpenSSL Project, 2024. OpenSSL 3.3 documentation; accessed 2026-07-16.
- [13] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [14] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- [15] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [16] Bruce Schneier and John Kelsey. Cryptographic support for secure logs on untrusted machines. In *7th USENIX Security Symposium*. USENIX Association, 1998.
- [17] SQLite Project. SQLite FTS5 extension, 2026. Official documentation; accessed 2026-07-16.
- [18] SQLite Project. Write-ahead logging, 2026. Official documentation; accessed 2026-07-16.
- [19] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [20] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2023.

A Implementation Evidence Map

Table 7: Primary implementation locations for claims in this paper.

Concern	Source	Evidence surface
Memory atomicity and lifecycle	aetnamem/memory.py	_atomic, remember, recall, forget, promote
Origin and trust policy	aetnamem/core/policy.py	source classification, initial status, forget gating, supersession selection
Extraction confidence	aetnamem/extract/rules.py	deterministic patterns and fixed confidence annotations
Record ranking	aetnamem/retrieve/rank.py	weight constants, tokenization, fallback overlap, deterministic tie break
SQLite, FTS, and audit	aetnamem/store/sqlite.py	transactions, WAL, candidate cap, schemas, event chain, incremental verification
Graph index	aetnamem/graph.py	extraction, lifecycle propagation, seeds, spread, merge proposals, partitions
Graph fusion and checkpoints	aetnamem/memory.py	weighted RRF, path evidence, external checkpoint documents
Forensic digest bindings	aetnamem/memory.py	retrieval_sha256 over retrieval evidence, content_sha256 on admission events
Recall-forensics harness	paper/scripts/run_recall_forensics_eval.py	stdlib-only reconstruction, eligible-turn re-execution, mutation trials, checkpoint boundary
Approval authority	aetnamem/actions/approval.py	exact-plan HMAC, nonce, TTL, constant-time comparison
Action transitions	aetnamem/actions/engine.py	plan construction, verification, execution, uncertainty, compensation, recovery
Filesystem boundary	aetnamem/actions/adapters/filesystem.py	root and symlink checks, snapshots, atomic write, postcondition and compensation
Service roles	aetnamem/service/app.py	loopback restriction, agent/reviewer routes, bearer-token role checks
macOS sealing	aetnamem/service/encrypted_db.py, aetnamem/service/secrets.py, aetnamem/service/__main__.py	Keychain secret, OpenSSL command, HMAC, checkpoint/seal hook, cleanup

The repository documentation further specifies the audit serialization in docs/audit-log-spec.md, action protocol in docs/guarded-actions.md, graph behavior in docs/graph-memory-design.md, and platform backup requirements in docs/data-storage-and-backup.md.

B Reference Algorithms

B.1 Governed remember

```
def remember(subject, message, origin):
    with sqlite.begin_immediate():
        episode = store_episode(message, origin)
        audit("episode.ingested", digest(message), origin)
        candidate = deterministic_extract(message, origin)
        if candidate is None:
            return episode

        trust = trust_tier(origin)
        status = active if trust_is_user(trust) else quarantined
        prior = active_same_slot(candidate.fact_key) if status == active else []
        record = insert_record(candidate, episode, trust, status, prior)
        supersede(prior, by=record) # never runs for quarantined input
        graph_changes = index_or_quarantine(record)
        audit(record_transition(status), record, graph_changes)
        return record
```

B.2 Recall

```
def recall(subject, query, use_graph=False, limit=10):
    records, fts = active_fts_plus_recent(subject, query, cap=200)
    graph_candidates = {}
```

```

if use_graph:
    seeds = graph_fts(query, cap=16)
    graph_candidates = spread_active_only(
        seeds, depth=2, frontier=64
    )
    records += active_records_nominated_outside_fts(graph_candidates)
direct = rank(records, text=0.75, trust=0.15, recency=0.10)
ranked = weighted_rrf(direct, graph_candidates) if use_graph else direct
returned = ranked[:limit]
ledger = ranked[:max(50, limit)]
audit_bounded_retrieval(query_digest, ledger, returned=returned)
return returned

```

B.3 Guarded commit

```

def commit(transaction):
    require(transaction.state == APPROVED)
    recompute_and_verify_world_patch(transaction)
    verify_unexpired_exact_plan_approval(transaction)
    verify_adapter_manifests(transaction)
    revalidate_all_preconditions(transaction)
    persist(COMMITTING)

    applied = []
    for operation in dependency_order(transaction):
        if not revalidate(operation):
            return compensate_and_finish(applied)
        persist(operation, EXECUTING)
        try:
            receipt = execute_without_db_transaction(operation)
        except BaseException:
            persist(operation, UNCERTAIN)
            compensate(applied)
            return finish(UNCERTAIN)
        if not verify_postcondition(operation, receipt):
            persist(operation, UNCERTAIN)
            compensate(applied)
            return finish(UNCERTAIN)
        persist(operation, VERIFIED, receipt_digest)
        applied.append(operation)
    return finish(COMMITTED)

```

These listings are explanatory reductions. The source files in Appendix A are authoritative and include validation and audit detail omitted here.

C Operational Recovery Notes

C.1 macOS

Quit the service normally so the current WAL state is checkpointed and sealed. Back up `memories.db.enc` and `memories.db.enc.hmac` together, plus the assistant workspace. Recovery on another Mac also requires the `aetname/database-key` Keychain value. Exporting that value displays a live secret and should occur only into an encrypted recovery process. If graph archive was explicitly enabled, back up its directory separately and encrypt the backup.

C.2 Linux and Windows

Stop the service before copying `memories.db`, or use a SQLite-aware online backup. Back up the workspace separately. Both database and backup destination are plaintext at the application layer, so use encrypted storage. On Windows, confirm BitLocker status and recovery-key custody. On Linux, confirm the data path is actually inside the intended encrypted volume; installing `cryptsetup` alone does not encrypt an existing home directory.

D Artifact and Authorship Statement

This document is a technical white paper prepared for arXiv-style distribution. It has not been peer reviewed. The author and requested attribution for the manuscript are **aetna000.com**. The core implementation artifact is AETNAMEM v0.3.0 at commit `69ebedea93ea95965867af5483dd9f63425b8e75`. The recall-forensics evidence uses a later development snapshot identified by the 42-file SHA-256 manifest embedded in `paper/data/recall-forensics.json`; its base commit alone is not the evaluated source. Both evidence JSON files record single local runs and should remain paired with their scripts and interpretation limits.

The paper intentionally distinguishes implemented behavior, deployment assumptions, measured evidence, design projections, and future work. Statements such as “groundbreaking,” “unbreakable,” “always encrypted,” “exactly once,” or “massive scale proven” are not supported by the current artifact and are deliberately not used.